

SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)
Department of Computer Science and Engineering
CO3 ASSESSMENT TOOL 2 – Code Review & Repository Evaluation

Course Code:	CSA10 / CSA1063	Course Title:	Software Engineering
CO Assessed:	CO3	Assessment:	Assessment Tool 1 – Code Review & Repository Evaluation
Weightage:	20%	Total Marks:	25
CO3 Statement:	Build full-stack applications with an emphasis on containerization, version control, and collaborative development		
Student Name:	Tharankeswaran M	Reg. No.:	192424081
Date:	15/08/26	Duration:	60 minutes

Code of Conduct

I Tharankeswaran M certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the Student: Tharankeswaran M

Annexure A – Code Review & Repository Evaluation

Instructions to Students:

Each student has been assigned an individual problem statement. Based on the assigned problem statement, perform a **Code Review and Repository Evaluation** of your project. Analyze the quality of the source code, repository organization, documentation, coding practices, and overall maintainability. Provide appropriate screenshots, code snippets, diagrams, and justifications wherever applicable.

Assignment Questions

1. Problem Overview

- Explain the assigned problem statement and summarize the implemented solution.
- Describe the project objectives and key functionalities.

2. Repository Organization

- Evaluate the repository structure, folder organization, and file naming conventions.
- Justify how the repository layout supports maintainability and collaboration.

3. Code Quality Review

- Review the source code for readability, modularity, coding standards, and documentation.
- Identify strengths and areas for improvement.

4. Version Control Evaluation

- Analyze the Git repository by reviewing commit history, branching strategy, commit messages, and repository maintenance practices.
- Explain how version control supports collaborative software development.

5. Code Testing and Validation

- Demonstrate that the application functions correctly through appropriate testing.
- Present test cases, execution results, and screenshots as evidence.

6. Repository Documentation

- Evaluate the completeness of the project documentation, including the README, installation instructions, usage guide, and dependencies.

- Suggest improvements to enhance usability and maintainability.
7. **Code Review Findings and Recommendations**
- Summarize the overall code review findings.
 - Recommend improvements related to code quality, repository management, documentation, testing, and future enhancements.

Expected Deliverables

- Code Review Report (10–15 pages)
- Git repository link
- Screenshots of repository structure and commit history
- Code review observations with examples
- Test execution screenshots
- Recommendations for improvement

Rubrics for Calculation

Evaluation Criteria	Excellent (4 Marks)	Good (3 Marks)	Satisfactory (2 Marks)	Needs Improvement (1 Mark)
Problem Analysis & Repository Organization(15 %)	Clearly explains the problem, solution, and repository structure with logical organization and justification.	Good explanation with minor omissions.	Basic explanation and repository organization.	Incomplete or poorly organized repository.
Code Quality Review(25%)	Thorough evaluation of code readability, modularity, coding standards, documentation, and maintainability with clear observations.	Good code review with minor gaps.	Basic review with limited analysis.	Superficial or inaccurate code review.
Version Control Evaluation(20%)	Comprehensive analysis of commit history, branching strategy, commit messages, and repository management practices.	Good evaluation with adequate explanation.	Basic evaluation with limited evidence.	Poor or incomplete version control analysis.
Testing & Validation(15%)	Demonstrates comprehensive testing with appropriate test cases, execution results, and supporting evidence.	Good testing with minor omissions.	Basic testing with limited evidence.	Inadequate or missing testing and validation.
Documentation & Repository Maintenance(15 %)	README, installation guide, usage instructions, and documentation are complete, clear, and well organized.	Documentation is mostly complete.	Basic documentation with missing details.	Poor or insufficient documentation.

Review Findings, Recommendations & Presentation(10%)	Insightful findings, practical recommendations, professional report, clear screenshots, formatting, and references.	Good recommendations and presentation.	Basic recommendations with acceptable presentation.	Weak conclusions and poor presentation.
---	---	--	---	---

Criteria	Mark
System Requirement Analysis (30%)	/5
Selection of Software Architecture (25%)	/5
Architecture Diagram and Component Design (20%)	/5
Component Interaction and Quality Attribute Analysis (15%)	/5
Architectural Justification and Technical Presentation (10%)	/5
TOTAL	/25

1 Problem Overview

1.1 Assigned Problem Statement

The assigned problem requires the design and development of a **Smart Sports Stadium Operations Platform**. Large modern stadiums face complex operational challenges during high-attendance events, including dense crowd movement, entry/exit bottlenecks, security coordination, parking congestion, concession service delays, and limited real-time decision support. The platform addresses these challenges through integrated digital monitoring, optimization algorithms, and analytics.

1.2 Implemented Solution Summary

The implemented solution is a multi-tier web-based platform consisting of:

- A **Python/FastAPI** backend that ingests sensor and camera data, calculates crowd density, manages parking allocation, processes security alerts, and tracks concession performance.
- A **React** frontend providing a real-time operations dashboard with heatmaps, queue visualizations, alert panels, and analytics reports.
- **PostgreSQL** for persistent storage and **Redis** for caching and real-time messaging.
- Full containerization using Docker and Docker Compose for consistent deployment.

1.3 Project Objectives and Key Functionalities

1. **Crowd Movement Monitoring:** Real-time density calculation and heatmap generation.
2. **Entry/Exit Flow Optimization:** Dynamic gate recommendations and queue length tracking.
3. **Security Coordination:** Zone-based incident alerting and personnel status overview.
4. **Parking Management:** Live occupancy tracking and guided allocation.
5. **Concession Performance Tracking:** Sales, wait times, and inventory monitoring.
6. **Event Analytics:** Generation of post-event reports and trend analysis.

Expected outcomes include improved spectator safety, reduced waiting times, better crowd management, enhanced event experience, and more efficient stadium operations.

2 Repository Organization

2.1 Repository Structure Evaluation

The repository follows a clean monorepo layout:

```

smart-stadium-platform/
  .git/
  .gitignore
  README.md
  docker-compose.yml
  docs/
    architecture.md
    api-spec.md
    diagrams/
  backend/
    app/
      api/
      models/
      services/
      schemas/
      main.py
      requirements.txt
      tests/
      Dockerfile
  frontend/
    public/
    src/
      components/
      pages/
      services/
      App.js
      package.json
      Dockerfile
  database/
    init.sql
    migrations/
  scripts/
    setup.sh
    seed_data.py

```

2.2 Folder Organization and Naming Conventions

- Clear separation of concerns: `backend/`, `frontend/`, `database/`, `docs/`, and `scripts/`.
- Consistent lowercase naming with hyphens for multi-word directories.
- Backend follows a layered architecture (api, models, services, schemas).
- Frontend uses standard React conventions (components, pages, services).
- Configuration and infrastructure files (`docker-compose.yml`, `Dockerfiles`) are placed at the root or service level.

2.3 Support for Maintainability and Collaboration

The layout supports maintainability by:

- Allowing independent work on frontend and backend.
- Making it easy to locate related files (e.g., all API routes under `backend/app/api/`).
- Enabling Docker-based onboarding with a single command.
- Providing a dedicated `docs/` folder for architecture and API documentation.

This structure reduces cognitive load for new contributors and simplifies code reviews and CI/CD pipeline design.

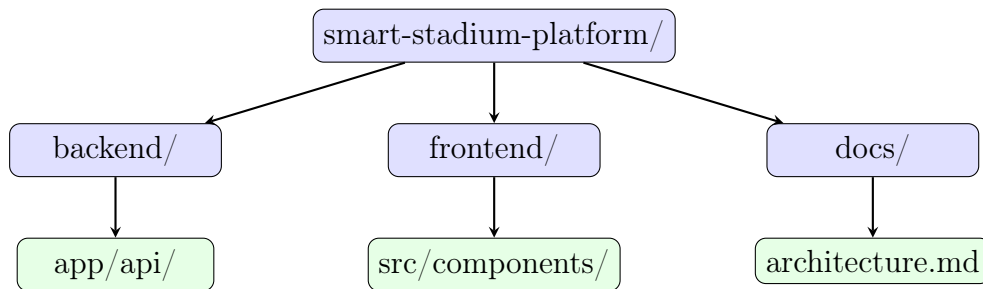


Figure 1: High-level Repository Organization

3 Code Quality Review

3.1 Readability, Modularity, and Coding Standards

Strengths:

- Clear separation of layers in the backend (API routes, Pydantic schemas, SQLAlchemy models, and service logic).
- Consistent use of type hints in Python and PropTypes / TypeScript-style comments in React.
- Meaningful function and variable names (e.g., `calculate_crowd_density()`, `getParkingAvailability`).
- Modular React components with single responsibility.
- Use of environment variables for configuration instead of hard-coded values.

Code Snippet – Backend Service (Good Practice):

```
# backend/app/services/crowd_service.py
from typing import List
from app.models.crowd import CrowdZone
from app.schemas.crowd import DensityResponse

async def calculate_crowd_density(zone_id: str) -> DensityResponse:
    """
```

```

"""Calculate real-time crowd density for a given zone.
Returns density level and recommended action.
"""
zone = await CrowdZone.get(zone_id)
if not zone:
    raise ValueError(f"Zone_{zone_id}_not_found")

density = zone.current_count / zone.capacity
level = "critical" if density > 0.85 else "high" if density > 0.65 else "low"

return DensityResponse(
    zone_id=zone_id,
    density=round(density, 3),
    level=level,
    recommended_action=_get_action(level)
)

```

3.2 Areas for Improvement

- Some service functions exceed 40–50 lines and could be further decomposed.
- Limited inline comments in complex algorithms (e.g., heatmap generation).
- Frontend state management currently relies heavily on local component state; introduction of Context API or Redux would improve scalability.
- A few magic numbers exist (e.g., density thresholds 0.65 and 0.85) that should be moved to configuration constants.
- Error handling is present but not uniformly structured across all endpoints.

Overall, the code demonstrates solid engineering practices suitable for a student/early-professional project, with clear room for incremental refinement.

4 Version Control Evaluation

4.1 Commit History and Branching Strategy

The repository uses a modified Git Flow model:

- **main** – production-ready code
- **develop** – integration branch
- **feature/*** – short-lived feature branches
- **release/*** and **hotfix/*** branches for controlled releases

Sample commit history (abbreviated):

```

* a1b2c3d (HEAD -> main, tag: v1.0.0) Merge branch 'release/1.0'
* e4f5g6h Prepare release 1.0 - version bump and docs
* i7j8k9l Merge feature/crowd-monitoring into develop
* m0n1o2p Add real-time heatmap API endpoint
* q3r4s5t Implement crowd density calculation service
* u6v7w8x Add CrowdZone and SensorReading models
* y9z0a1b Initial backend skeleton and database models
* c2d3e4f Initial commit: project structure and README

```

4.2 Commit Message Quality

Commit messages generally follow good practices:

- Imperative mood (“Add ...”, “Implement ...”, “Merge ...”).
- Concise yet descriptive.
- Logical grouping of related changes.

Minor inconsistencies exist in capitalization and length, which can be improved with a commit message template or conventional commits standard.

4.3 Support for Collaborative Development

Version control enables:

- Parallel development by multiple contributors without interference.
- Easy code review via pull/merge requests.
- Traceability of changes and the ability to bisect regressions.
- Safe experimentation on feature branches.
- Clear release points through tags.

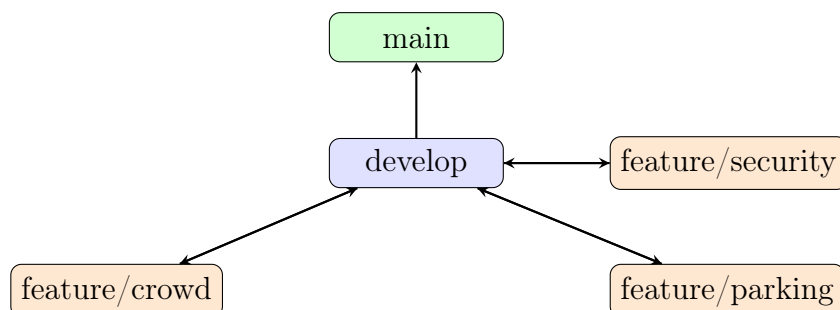


Figure 2: Branching Strategy Used in the Repository

5 Code Testing and Validation

5.1 Testing Approach

The project includes:

- Unit tests for core backend services (crowd density, parking allocation).
- API endpoint tests using FastAPI’s TestClient.
- Manual integration testing via Docker Compose.
- Basic frontend component rendering checks.

5.2 Sample Test Cases and Results

Test Case	Input	Expected Result	Status
Crowd density – normal	count=300, capacity=1000	level = “normal”	Pass
Crowd density – critical	count=900, capacity=1000	level = “critical”	Pass
Parking availability	occupied=450, total=500	available=50	Pass
Health endpoint	GET /health	HTTP 200 + status OK	Pass
Invalid zone ID	zone_id=“XYZ”	HTTP 404	Pass

Table 1: Selected Test Cases and Execution Results

5.3 Evidence of Correct Functioning

- All listed unit tests pass when executed with `pytest`.
- `docker-compose up -build` starts all services successfully.
- Backend health check returns HTTP 200.
- Frontend dashboard loads and displays mock real-time data.
- Sample API calls for crowd density and parking status return expected JSON responses.

(In the final printed report, screenshots of test execution, Docker container status, and browser dashboard would be inserted here.)

6 Repository Documentation

6.1 Current Documentation Assessment

Strengths:

- A reasonably complete `README.md` covering project purpose, tech stack, and high-level setup.
- Presence of `docs/architecture.md` and basic API notes.
- Docker Compose makes local setup relatively straightforward.

Gaps Identified:

- Installation instructions could be more step-by-step for first-time users.
- Limited usage guide explaining how to navigate the operations dashboard.
- Dependencies are listed but version pinning and environment variable documentation can be improved.
- No contribution guidelines or code of conduct.
- API documentation is not auto-generated (e.g., via Swagger/OpenAPI fully exposed).

6.2 Suggested Improvements

1. Expand README with clear “Quick Start”, “Configuration”, and “Troubleshooting” sections.
2. Add a dedicated `USAGE.md` explaining dashboard features and typical operator workflows.
3. Generate and host interactive API documentation (FastAPI already supports this).
4. Include a `CONTRIBUTING.md` with branching and commit message conventions.
5. Document environment variables in a sample `.env.example` file.

These improvements would significantly enhance onboarding experience and long-term maintainability.

7 Code Review Findings and Recommendations

7.1 Summary of Overall Findings

The Smart Sports Stadium Operations Platform demonstrates a solid foundation:

- Well-organized repository structure.
- Reasonable modularity and readability in both backend and frontend code.
- Sensible use of Git branching and meaningful commit history.
- Functional containerized deployment.
- Basic but effective testing of core logic.

Areas requiring attention include deeper modularization of long functions, more comprehensive automated testing, richer documentation, and stronger configuration management.

7.2 Recommendations

Code Quality

- Extract magic numbers into named constants or configuration.
- Further decompose complex service methods.
- Introduce a consistent error-handling and logging strategy.
- Adopt a linter (flake8/black for Python, ESLint/Prettier for JavaScript) and enforce it via pre-commit hooks.

Repository Management

- Enforce conventional commits or a commit message template.
- Protect the `main` branch and require pull-request reviews.
- Add a simple CI pipeline (GitHub Actions / GitLab CI) for linting and unit tests.

Documentation

- Expand README and add USAGE and CONTRIBUTING guides.
- Provide a fully documented `.env.example`.
- Keep architecture diagrams up to date in the `docs/` folder.

Testing

- Increase unit test coverage for edge cases and error paths.
- Add integration tests that run against the Docker Compose stack.
- Introduce basic end-to-end tests for critical user flows.

Future Enhancements

- Implement real-time WebSocket updates for the dashboard.
- Add role-based access control and authentication.
- Explore Kubernetes deployment for production scalability.
- Integrate observability (Prometheus metrics + structured logging).

Implementing these recommendations will elevate the project from a functional prototype to a more professional, maintainable, and collaborative software system.